Rebu Coding Standards

Introduction:

This document is a comprehensive guide to the best coding standards and practices for ReBu jobs, built with a *React* frontend, *Django* backend, and *PostgreSQL* database hosted on *Supabase*. All contributors are expected to follow these standards to maintain code quality, readability, and consistency across the codebase.

General Principles:

- Readability and Clarity: Code should be easily understandable by any team member. Prioritize clarity over shorthands or clever naming.
- Consistency: The same conventions should be applied consistently within each part of the stack. When in doubt, follow the existing patterns in the codebase, or consult this document.
- Simplicity: Write simple, efficient code. Avoid premature optimization and unneeded abstractions.
- Documentation: Code should be well commented where the logic is not immediately obvious. Every file should make its purpose clear.

Formatting:

Indentation and Whitespace:

- Use 2 spaces for indentation in all frontend files (.js, ,jsx, .css). This aligns with standard React community conventions.
- Use 4 spaces for indentation in backend files. This aligns with Python's PEP 8 standard.
- Use blank lines sparingly with functions to separate logical groups of statements.
- Remove all trailing whitespace.

Semicolons (Frontend):

- Always use semicolons at the end of statements in JavaScript/JSX files.

Braces:

- Opening braces should be done on the same line as the statement (K&R style) in both frontend and backend code.
- Always use braces for if, else, for, and while blocks, even if the body is a single statement.

Line Length:

- Lines should not exceed 100 characters (where practical).
- Exception | Inline style objects in JSX may exceed this limit. This is acceptable to avoid fragmenting style definitions across many lines. However, if a style constant is reused across components, extract it into a constant.

Quotes:

- Use double quotes for JSX attribute values and string literals in JavaScript.

- Use single quotes for import statements ONLY.

Naming Conventions:

- **Frontend (React):**

Element	**Convention**	**Example**
Components	PascalCase	JobBoard
Functions/Methods	camelCase	handleLogin
Variables	camelCase	hoveredJob
Constants (config/static)	camelCase or UPPER_SNAKE_CASE	statusColors or MAX_RETRIES
Boolean variables	Prefixed with *is, had, should*	hasError, isLoading
Event handlers	Prefixed with *handle*	handleLogin, handleSignup
State setters	Prefixed with *set* (via useState)	setRole, setSearchQuery
CSS class names	kebab-case	app-header, job-card
File names (components)	PascalCase with *.jsx* extension	JobBoard.jsx, LoginPage.jsx
File name (utilities)	camelCase with *.js* extension	reportWebVitals.js, setupTests.js

- **Backend (Django):**

Element	**Convention**	**Example**
Classes	PascalCase	JobRequest, UserProfile
Functions/Methods	snake_case	get_job_by_id, create_user
Variables	snake_case	job_title, search_query
Constants	UPPER_SNAKE_CASE	MAX_RETRIES, DEFAULT_PAGE_SIZE
File names	snake_case with *.py* extension	job_views.py, user_models.py
URL Patterns	kebab-case	/api/job-requests/, /api/user-profile/

- **Database (PostgreSQL/Supabase):**

Element	**Convention**	**Example**
Table names	snake_case, plural	job_requests, user_profiles
Column names	snake_case	created_at, posted_by
Primary keys	*id*	id
Foreign keys	Referenced table (singular) + *_id*	user_id, job_request_id
Index names	*idx_* + table + _ + column	idx_job_requests_status

Code Structure:

- File organization (Frontend)
  - frontend/src/
    - components/ || Reusable UI Components
    - pages/ || Full page components (one per route)
    - utils/ || Helper functions and constants
    - App.jsx || Root component with routing
    - index.js || Entry point
    - index.css || Global styles
  - Each page component gets its own *.jsx* file named in PascalCase.
  - Shared/reusable components (e.g., a Header or a Button component) should be extracted into the *components/* directory.
  - Utility functions and shared constants should live in *utils/*.

Component Structure:

- Within a React file component, organize the code in this order:
  1. Imports - external libraries first, then local imports
  2. Constants - any static data or configuration used by the component
  3. Helper Components - small sub components used only within this file
  4. Main Component - the default exported component
  5. Export - default export at the bottom (or inline with the function declaration)

Functions and methods

- Functions should be short and perform a single task.
- Keep parameter count to a minimum. If a function needs more than three parameters, consider using an object parameter.
- Use arrow functions for inline callbacks and helper components. Use regular function declarations for main exported components.

Comments and Documentation:

When to Comment:

- **API Integration points:** Mark every place where the frontend expects data from the backend with a clear comment block describing the endpoint, method, and expected data shape.
- **TODOs**: Use *// TODO* for planned work. Include relevant Jira ticket number when applicable.
- **Complex logic**: Explain any logic that is not immediately obvious from the code.
- **Section separations**: Use comment banners to separate major sections within large component files.

When NOT to Comment:

- Do not comment obvious code (e.g., *// set loading to true* above *setLoading(true)*).
- Do not leave commented-out code in the codebase long term. If code is no longer needed, delete it. Git history will preserve old code.

Version Control (Git):

Branching:

- Create a new branch for each Jira story or task.
- Branch names should follow the format: *TM13-{ticket number}-{brief-description}*
    - Ex. *TM13-20-Create-worker-and-client-view-of-worker-ratings*
- Do NOT commit directly to main.

Commits:

- Write meaningful commit messages that describe what changed and why.
- Commit related changes together as a logical unit.

Pull Requests:

- All code must be merged into main via a Pull Request.
- PRs should be reviewed by at least one other team member before merging.
- Keep PRs focused. One feature or fix per PR.

Error Handling:

Frontend:

- Use try/catch blocks for all API calls.
- Display user friendly error messages via toasts or inline error states.
- Never silently swallow errors, at minimum, log them to the console.

Backend:

- Use Django's built in exception handling and return appropriate HTTP status codes.
- Return consistent JSON error responses.

Testing:

Testing:

- Write unit tests for new components and utility functions where practical.
- Test files should be co-located with the code they test or placed in a __tests__ directory.
- Test file naming: ComponentName.test.js for frontend, test_module.py for backend.
- Prioritize meaningful tests over high coverage numbers.

Security:

Security:

- NEVER commit API keys, secrets, or credentials to the repository. Use environment variables, and .env files (which must be listed in .gitignore).
- Use Django's built in protections against CSRF, XSS, and SQL injection.
- Validate and sanitize all user input on both frontend and backend.
- Use parametrized queries or Django ORM. Never construct raw SQL strings.

Conclusion:

This document establishes the coding standards for the ReBu project. All team members are expected to follow these conventions to maintain a clean, consistent, and maintainable codebase. When encountering a situation not covered by this document, follow the conventions already established in the existing code and discuss with the team.